Below is a PRD you can drop into a doc for Antigravity. It's written feature-first, not implementation-first, so design + dev can align.

QUILL – Programmable Intent Layer for Digital Rupee

Product Requirements Document (PRD)

1. Product Overview

1.1 Vision

QUILL is a **user-programmable protocol layer** that turns the Digital Rupee into a **trust-automation tool**. It lets:

- **Individuals** define and enforce personal spending rules (intents) like "Cap Swiggy at ₹10,000/month" without code.
- **Small businesses** define controlled budgets for employees, with multi-admin approval and automatic enforcement so funds are not diverted from their intended purpose.

Behind the UI, QUILL exposes an API that any UPI/Digital Rupee payment app can call to screen a transaction in real time against the user's intents and decide whether to allow, warn, or block.[ppl-ai-file-upload.s3.amazonaws]

1.2 Primary Users

- **Individual user**: wants personal guardrails on spending, simple insights, and alerts.
- **Small Business**:
  - **Client Admin** (CEO + up to 4 more admins): defines budgets/intents, approves allocations, oversights usage.
  - **Business User** (employees/contractors): requests budget, spends against it, provides short justification.

1.3 Platforms

- **Web app** (desktop & tablet).
- **Mobile app** (responsive web baseline + room for native later).

Same backend for both.

---

2. Core Concepts

2.1 Intent

An **Intent** is a rule that says how a specific pot of money can be used:

- For individuals: "Swiggy/Zomato spend ≤ ₹10,000 per month; warn at 50%, block beyond 100%."

- For SMEs: “Employee X gets ₹50,000 to buy AI tools in March, only at approved vendors, must be dual-approved by 2 client admins.”

INTENT = {who, where, when, what_for, how_much, what_to_do_if_exceeded}

2.2 QUILL as a payment option

When paying on Swiggy/any merchant:

- User sees **“Pay via QUILL (Digital Rupee)”** as a UPI-style payment method.
- Swiggy → calls QUILL’s /evaluate API with transaction details and the user’s QUILL ID.
- QUILL checks active intents and replies:
    - allow → payment continues.
    - warn_and_allow → show warning but let user proceed.
    - block / require_approval → payment prevented or routed for approval.

For hack/demo: simulate this as a “merchant checkout” page embedded in QUILL.

---

3. User Flows – Individual

3.1 Sign Up / Login

Flow:

1. User lands on QUILL web/mobile.
2. Clicks **Sign up**:
    - Inputs: Email, password.
    - OTP or email verification (optional for hack, but design for it).
3. After signup, system asks:
    - “What type of account is this?”
        - Option A: Individual
        - Option B: Small Business
4. If Individual selected → go to Individual Onboarding.

3.2 Individual Onboarding

- Collect:
    - Name
    - Optional monthly income range (for insights)
- Show toggle: light / dark mode (global theme).
- Drop them onto **Individual Dashboard**.

3.3 Individual Dashboard – Features

Sections:

1. **Top summary cards:**
    - "Total Spending This Month"
    - "Top 3 categories"
    - "Active Intents (N)"
    - "Alerts this month"
2. **Spending analytics:**
    - Category pie chart (Food, Shopping, Travel, etc.).
    - Timeline chart (daily/weekly spend).
3. **Intent list:**
    - Cards for each active intent:
        - Title (e.g., "Swiggy/Zomato Cap – ₹10,000")
        - Status (OK / 50% reached / 100% reached)
        - Progress bar
        - Button: "View details / Edit / Pause"
4. **"Create Intent" panel (LLM prompt box):**
    - Text area: "Describe how you want your money to behave."
    - Example placeholders:
        - "Set a monthly limit of ₹10,000 for Swiggy and Zomato."
        - "Warn me when I've spent ₹5,000 on Amazon this month."
    - Button: **Generate Intent**.

3.4 Individual: Create Intent (Swiggy/Zomato use case)

User Story:

As an individual, I want to cap Swiggy/Zomato at ₹10,000/month, be alerted at 50%, and block further payments beyond 100%.

Flow:

1. User types:
    - "Set limit for Swiggy and Zomato to ₹10,000 per month, warn me at ₹5,000."
2. QUILL calls LLM → converts to internal policy:
    - Targets: merchants Swiggy, Zomato (UPI IDs or tags).

    - Monthly cap: 10000.
    - Warning threshold: 5000.
    - Actions:
        - ≤ 5000: allow.
        - 5000–10000: warn_and_allow.
        - 10000: block.
3. UI shows:
    - Human summary:
        - "You're capping Swiggy/Zomato spend at ₹10,000/month. Warning after ₹5,000."
    - Technical preview (collapsed): JSON policy.
4. User clicks **Activate**.

3.5 Individual: How QUILL tracks & enforces

Assumption: QUILL sees transactions either via:

- (A) Direct integration (merchant / bank calls /evaluate before debit), or
- (B) Connected account / aggregator (for demo: simulate inside QUILL).

Flow when user pays on Swiggy:

1. User selects **Pay via QUILL** at checkout (mocked).
2. Swiggy sends:

```
json
{
  "quill_user_id": "U123",
  "merchant": "SWIGGY",
  "amount": 1200,
  "category": "food_delivery",
  "timestamp": "...",
  "payment_method": "digital_rupee"
}
```

3. QUILL:
    - Finds active intents for user matching merchants Swiggy/Zomato.
    - Calculates month-to-date spend for that intent.

    - Adds amount → check thresholds.
    - Returns:

```
json
{
  "outcome": "warn_and_allow",
  "ui_message": "You've reached 60% of your Swiggy/Zomato limit (₹6,000 of ₹10,000).",
  "new_total": 6000
}
```

4. Swiggy (UI demo) shows warning; if user confirms, payment proceeds.
5. For a transaction that exceeds 10K, QUILL returns block with message: “Swiggy/Zomato limit of ₹10,000/month exceeded.”
6. Alerts:
    - At 50% -> push/email in QUILL: “Half of your Swiggy budget used.”
    - At 100% -> “Swiggy/Zomato limit reached for this month.”

3.6 Individual: Stats & Graphs

For each intent:

- Progress card (donut chart showing used vs remaining).
- Timeline showing spend over the month.
- Breakdown by merchant (Swiggy vs Zomato etc.).

Global:

- Total intents vs triggered warnings vs blocks (bar/line charts).

---

4. User Flows – Small Business

4.1 Small Business Onboarding

After signup → selects “Small Business”.

Steps:

1. Business profile:
    - Business name
    - Sector (dropdown)
2. Company size selection:
    - Radio buttons: 1–10, 11–20, 21–30, 31–50 employees.

3. Define **Client Admins**:
    - CEO is default first Client Admin.
    - Can invite up to 4 more by email (max 5 client admins total).
4. All users by default start as “Business Users”.
5. Client Admins get admin UI; others get user UI after login.

4.2 Roles

- **Client Admin**:
    - Create and edit business intents (budgets).
    - Approve allocations (2 client admins must approve).
    - View all dashboards and reports.
- **Business User**:
    - Request allocations (“I need ₹X for Y tool”).
    - See their own allocated budgets and spend.
    - Provide one-line purchase justification.

4.3 Small Business – Client Admin Dashboard

Sections:

1. **Top cards:**
    - Total monthly company budget.
    - Allocated vs unallocated.
    - Number of open user requests.
    - Active intents count.
2. **Team & roles:**
    - List of Client Admins.
    - List of Users (basic view).
3. **Budget & Intents:**
    - List of project/usage-based budgets:
        - “AI Tool Subscriptions – ₹1,00,000”
        - “Cloud Credits – ₹2,00,000”
    - Each has:
        - Available amount, used amount.
        - Linked intents (rules on where/how can be spent).

4. **Copilot Intent Suggestions (optional in PRD; nice-to-have):**
    - Cards showing recommended intents based on past spend.

4.4 Small Business – Client Admin: Create AI Tool Budget Intent (via prompt)

User Story:

As a Client Admin, I want to set up a controlled budget for AI tool subscriptions so staff can only use the funds for approved tools, with dual admin approval.

Flow:

1. Admin clicks **Create Intent**.
2. Textbox prompt example:

"Create a budget of ₹50,000 for AI tools this month. Only allow vendors OpenAI, Anthropic, GitHub, HuggingFace. Every allocation to employees must be approved by 2 client admins."

3. QUILL LLM parses into an Intent:
    - Scope: wallet = "AI Tools Budget", type=project.
    - Conditions:
        - allowed_categories: [ai_subscription, developer_tool].
        - allowed_recipients: list of vendor IDs.
        - monthly_cap: 50000.
    - Actions:
        - on_match: allow.
    - Risk rules:
        - off_intent: require_approval or block.
    - Additional business rule: any **allocation** from this budget to a user requires 2 client admin approvals.
4. UI shows summary:
    - "Budget: ₹50,000 for AI tools in March. Vendors: OpenAI, Anthropic, GitHub, HuggingFace. Allocations require dual admin approval."
5. Admin clicks **Activate**.

4.5 Business User: Request Allocation

User Story:

As a Business User, I can request some of the AI Tool Budget via a simple prompt, instead of filling complex forms.

Flow:

1. User goes to **Requests** tab.

2. Textbox: “Describe what you need and why.”
3. Example input:
    - “I need ₹4,000 for a ChatGPT Plus subscription for 1 month for client project X.”
4. QUILL LLM extracts:
    - Amount: 4000.
    - Purpose: ai_subscription.
    - Suggested intent link: AI Tools Budget.
5. QUILL creates a **Pending Allocation** record with:
    - User ID, amount, purpose tag, linked budget.
    - Status: pending_approval.
6. All Client Admins receive notification:
    - “User A requested ₹4,000 from AI Tools Budget for ChatGPT Plus.”

4.6 Client Admin: Dual Approval Flow

Requirements:

- Every allocation must be approved by at least **two Client Admins**.

Flow:

1. In Client Admin panel, under **Requests**:
    - Each request card shows:
        - User, amount, purpose, suggested budget (intent).
        - Current approvals: 0/2, 1/2, 2/2.
2. First Admin clicks “Approve” (or Reject).
    - If Reject, status = rejected; user notified.
    - If Approve, approvals count increments.
3. Second Admin must also Approve:
    - When approvals reach 2/2 → allocation status = approved.
4. Once approved:
    - Allocation becomes an **Intent Contract from company wallet to that user**:
        - For that user, this becomes a “User-level Intent”:
            - Amount limit: approved amount.
            - Purpose/category: ai_subscription.
            - Window: e.g., 30 days.

        - Merchants: allowed vendor list.

User sees it on their **User Dashboard** as “Allocated budget: AI Tools – ₹4,000 (valid till <date>)”.

4.7 Business User: Spending with QUILL

User Story:

As a Business User, I want QUILL to automatically verify my AI-tool purchase matches the approved budget and purpose.

Flow:

1. User goes to vendor (e.g., OpenAI) and checks out.
2. At payment, selects **Pay via QUILL**.
3. Vendor sends to QUILL:

```
json
{
  "quill_user_id": "User123",
  "merchant": "OPENAI",
  "amount": 3900,
  "category": "ai_subscription",
  "timestamp": "...",
  "description": "ChatGPT Plus monthly"
}
```

4. QUILL:
    - Finds active user-level intent(s) for AI Tools.
    - Checks:
        - Merchant in allowed list?
        - Category matches ai_subscription?
        - Amount ≤ remaining allocation?
    - If valid:
        - outcome: allow.
    - If invalid:
        - e.g., merchant not in list or category not matching:
            - outcome: block with message: “This purchase does not match AI tools intent.”
5. User is prompted to enter a **one-line description** in QUILL UI before final approval:

    - “What are you buying?” – they type: “ChatGPT Plus for client ABC project”.
6. QUILL logs description and confirms.

4.8 Post-Purchase Notifications & Remainder Handling

After successful purchase:

- QUILL updates:
    - Allocation used amount.
    - Remaining amount.
- Notifications to all Client Admins:
    - “User A used ₹3,900 of the AI Tools budget; ₹100 remaining.”
- In User Dashboard:
    - Shows the specific purchase, remaining amount, and that the intent is closed or still open (if there is remaining time/amount).

Optionally:

- If remaining amount is small or expired by date, QUILL can:
    - Auto-release back to company (clawback on expiry), or
    - Leave for future AI tool charges (configurable).

4.9 Business Dashboards (Admin & User)

Client Admin Dashboard:

- Graphs:
    - Budget allocation vs usage per category (AI tools, cloud, travel, etc.).
    - Per-employee usage.
    - Pie of “On-intent vs Off-intent attempts”.
- Tables:
    - All allocations, statuses, and dual-approval history.
    - All transactions with compliance flag (OK / blocked reason).

Business User Dashboard:

- Allocations list:
    - For each: budget name, total, used, remaining, validity.
- Request history:
    - Pending / Approved / Rejected.

---

5. Common Features / Cross-Cutting

5.1 Theming

- **Light / Dark mode toggle** (persist per user).
- Accessible color choices for warnings/blocks.

5.2 Notifications

- Channels:
    - In-app notifications (top bell icon).
    - Email (for critical events).
- Individuals:
    - 50% & 100% threshold alerts per intent.
- Business:
    - New allocation requests to all Client Admins.
    - Dual approval completion.
    - Off-intent blocked attempts.

5.3 LLM/NLP Prompting

Every role uses prompts to reduce friction:

- Individual:
    - "Set limit for Swiggy to 10k."
- Client Admin:
    - "Create AI tools budget with 50k monthly cap and dual admin approval."
- Business User:
    - "I need 4k for ChatGPT Plus for project X."

LLM engine:

- Extracts: purpose, amount, time window, merchants, category, required approvals.
- Fills policy object.
- UI shows summary + underlying rule for user confirmation.

---

6. Non-Functional & Constraints (for hackathon scope)

- MVP can simulate:
    - "Pay via QUILL" as a mock checkout page (no real Swiggy integration).
    - Risk engine can be basic (or omitted) due to time.

- Focus on:
    - Clean role flows,
    - Intent creation via prompts,
    - Evaluation + enforcement,
    - Graphs for at least one individual intent and one SME budget.

This PRD captures the flows you described (individual + small business, admin + user), connects them to a **user-programmable protocol layer** with LLM-driven intents and automated checks at payment time, and leaves enough room for the team to implement within hackathon constraints.[ppl-ai-file-upload.s3.amazonaws]